Hochschule Esslingen
University of Applied Sciences

OBJEKTORIENTIERTE SYSTEME

BEZIEHUNGEN ZWISCHEN OBJEKTEN IN C++

PROF. DR. JÖRG NITZSCHE
HOCHSCHULE ESSLINGEN
JOERG.NITZSCHE@HS-ESSLINGEN.DE

LERNZIEL

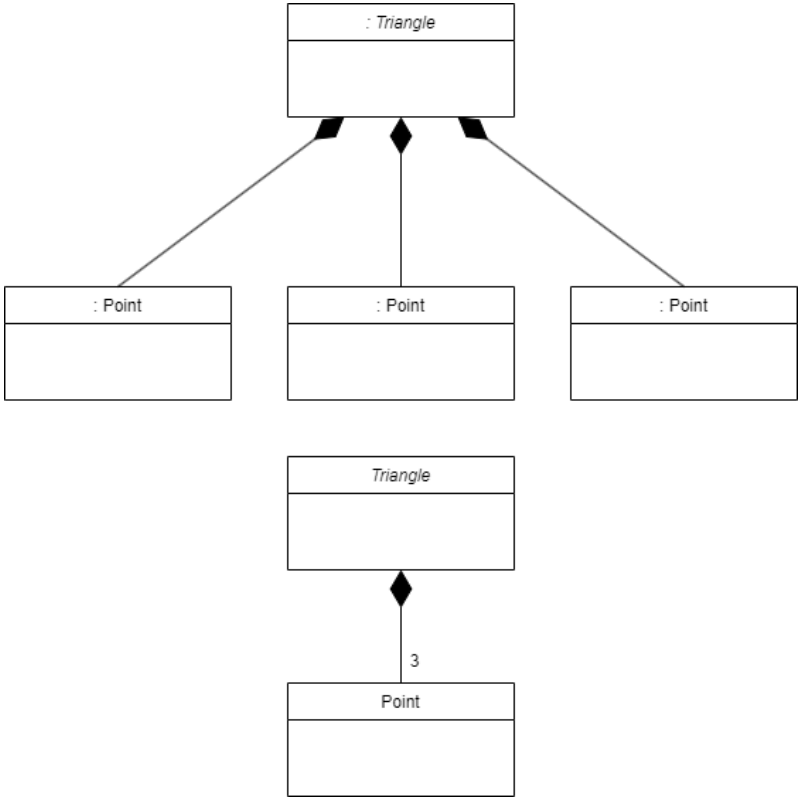
- Verstehen wie man Klassen bzw. Objekte über Attribute miteinander in Beziehung bringen kann.

1 Komposition

2 Aggregation

AUFGABENSTELLUNG

- Die Klasse **Triangle** setzt sich aus 3 Objekten der Klasse **Point** zusammen.
- Definieren Sie einen Konstruktor mit Parametern und die print() Methode der Klasse **Triangle.**
- Die Klasse **Point** wird als bekannt vorausgesetzt (siehe nächste Folie).



KLASSE POINT

```
#include <iostream>
using namespace std;

class Point {
  double _x, _y;
public:
  Point() {
    _x = 0;
    _y = 0;
    cout << "StandardKonstruktor von: "
         << _x << ", " << _y << endl;
  }
  Point(double x, double y) {
    this->_x = x;  this->_y = y;
    cout << "Konstruktor von: " << _x
         << ", " << _y << endl;
  }
  ~Point(){
    cout << "Destruktor von " << _x
         << ", " << _y << endl;
  }
  double get_x() const { return _x; }
  double get_y() const { return _y; }
  void set_x(double x) { this->_x = x; }
  void set_y(double y) { this->_y = y; }
  void print() {
    cout << "x= " << _x << endl;
    cout << "y= " << _y << endl; }
};
```

KLASSE TRIANGLE

```
1  class Triangle{
2    Point _p1;
3    Point _p2;
4    Point _p3;
5  public:
6    Triangle(double x1, double y1,
7             double x2, double y2,
8             double x3, double y3)
9    {
10     _p1.set_x(x1);
11     _p1.set_y(y1);
12     _p2.set_x(x2);
13     _p2.set_y(y2);
14     _p3.set_x(x3);
15     _p3.set_y(y3);
16   };
```

```
17   void print() {
18     _p1.print();
19     _p2.print();
20     _p3.print();
21   }
22 };
23
24 int main()
25 {
26     cout << "main beginnt" << endl;
27     {
28         cout << "Block beginnt" << endl;
29         Triangle t(-3, 0, 0, 4, 3, 0);
30         t.print();
31         cout << "Block endet" << endl;
32     }
33     cout << "main endet" << endl;
34     return 0;
35 }
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

Particify - QUIZ

```
1  #include <iostream>
2  using namespace std;
3
4  class Point {
5    double _x, _y;
6  public:
7    Point() {
8      _x = 0;
9      _y = 0;
10     cout << "StandardKonstruktor von: "
11          << _x << ", " << _y << endl;
12   }
13   Point(double x, double y) {
14     this->_x = x;  this->_y = y;
15     cout << "Konstruktor von: " << _x
16          << ", " << _y << endl;
17   }
```

```
18   ~Point(){
19     cout << "Destruktor von " << _x
20          << ", " << _y << endl;
21   }
22   double get_x() const { return _x; }
23   double get_y() const { return _y; }
24   void set_x(double x) { this->_x = x; }
25   void set_y(double y) { this->_y = y; }
26   void print() {
27     cout << "x= " << _x << endl;
28     cout << "y= " << _y << endl; }
29 };
```

Particify - QUIZ

```
1  class Triangle{
2    Point _p1;
3    Point _p2;
4    Point _p3;
5  public:
6    Triangle(double x1, double y1,
7             double x2, double y2,
8             double x3, double y3)
9    {
10     _p1.set_x(x1);
11     _p1.set_y(y1);
12     _p2.set_x(x2);
13     _p2.set_y(y2);
14     _p3.set_x(x3);
15     _p3.set_y(y3);
16   };
```

```
17   void print() {
18     _p1.print();
19     _p2.print();
20     _p3.print();
21   }
22 };
23
24 int main()
25 {
26     cout << "main beginnt" << endl;
27     {
28         cout << "Block beginnt" << endl;
29         Triangle t(-3, 0, 0, 4, 3, 0);
30         t.print();
31         cout << "Block endet" << endl;
32     }
33     cout << "main endet" << endl;
34     return 0;
35 }
```

Articify - QUIZ: AUFLÖSUNG

Ausgabe

```
main beginnt
Block beginnt
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
x= -3
y= 0
x= 0
y= 4
x= 3
y= 0
Block endet
Destruktor von 3, 0
Destruktor von 0, 4
Destruktor von -3, 0
main endet
```

```
17  void print() {
18      _p1.print();
19      _p2.print();
20      _p3.print();
21    }
22  };
23
24  int main()
25  {
26      cout << "main beginnt" << endl;
27      {
28          cout << "Block beginnt" << endl;
29          Triangle t(-3, 0, 0, 4, 3, 0);
30          t.print();
31          cout << „Block endet" << endl;
32      }
33      cout << "main endet" << endl;
34      return 0;
35  }
```

Particify - QUIZ: AUFLÖSUNG

```
1  class Triangle{
2    Point _p1;
3    Point _p2;
4    Point _p3;
5  public:
6    Triangle(double x1, double y1,
7             double x2, double y2,
8             double x3, double y3)
9    {
10     _p1.set_x(x1);
11     _p1.set_y(y1);
12     _p2.set_x(x2);
13     _p2.set_y(y2);
14     _p3.set_x(x3);
15     _p3.set_y(y3);
16   };
```

```
17 void print() {
18     _p1.print(
19     _p2.print
20     _p3.print
21   }
22 };
23
24 int main()
25 {
26     cout << "
27     {
28         cout
29         Triar
30         t.pri
31         cout
32     }
33     cout << "
34     return 0;
35 }
```

Ausgabe

```
main beginnt
Block beginnt
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
x= -3
y= 0
x= 0
y= 4
x= 3
y= 0
Block endet
Destruktor von 3, 0
Destruktor von 0, 4
Destruktor von -3, 0
main endet
```

Orticify - QUIZ: AUFLÖSUNG

```
cla
```

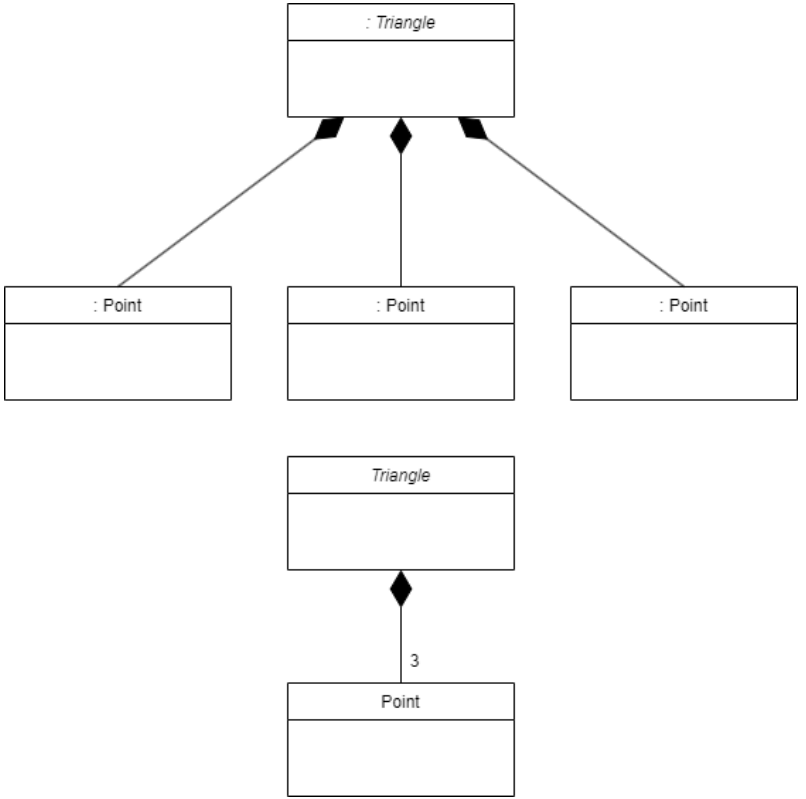
Ausgabe

```
main beginnt
Block beginnt
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
StandardKonstruktor von: 0, 0
x= -3
y= 0
x= 0
y= 4
x= 3
y= 0
Block endet
Destruktor von 3, 0
Destruktor von 0, 4
Destruktor von -3, 0
main endet
```

```
17  void print() {
18      _p1.print();
19      _p2.print();
20      _p3.print();
21    }
22  };
23
24  int main()
25  {
26      cout << "main beginnt" << endl;
27      {
28          cout << "Block beginnt" << endl;
29          Triangle t(-3, 0, 0, 4, 3, 0);
30          t.print();
31          cout << "Block endet" << endl;
32      }
33      cout << "main endet" << endl;
34      return 0;
35  }
```

KOMPOSITION

- Die Klasse **Triangle** setzt sich aus 3 Objekten der Klasse **Point** zusammen.
- Die Existenz der Punkte ist vom Dreieck abhängig!
  - Verschwindet das Dreieck, verschwinden auch die Punkte.
- Dies ist ein Beispiel für eine Komposition (part-of-Beziehung):
  - die Existenz der Teilobjekte ist vom Gesamtobjekt abhängig.

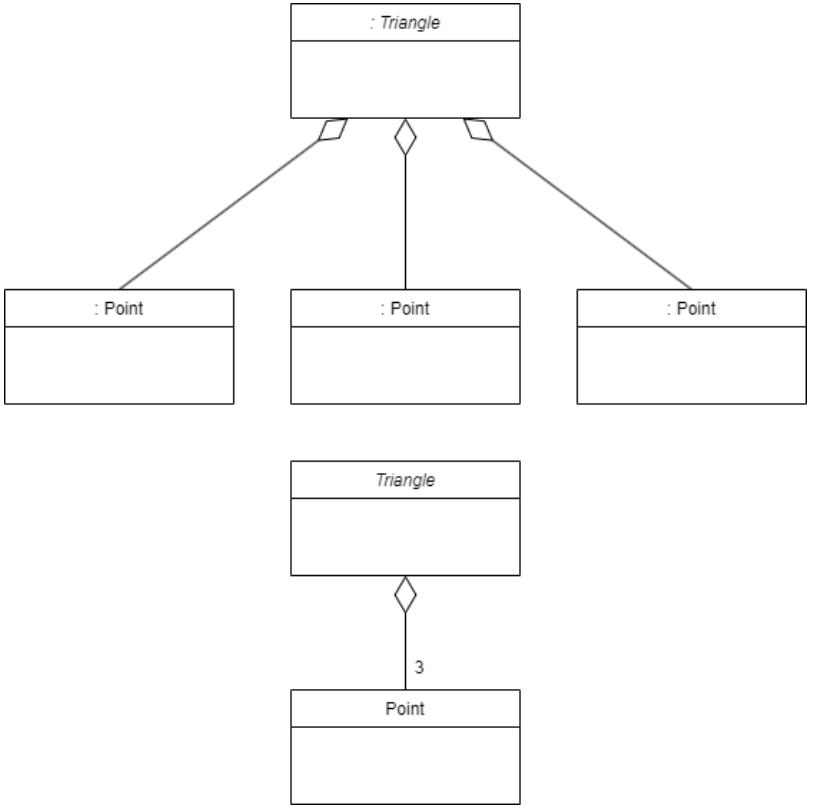


1 Komposition

2 Aggregation

AGGREGATION

- Eine Aggregation ist ebenfalls eine part-of-Beziehung. Dabei haben aber die Teilobjekte eine vom Gesamtobjekt unabhängige Existenz.
- Aufgabenstellung
  - Die Klasse **Triangle** setzt sich aus 3 Objekten der Klasse **Point** als *Aggregation* zusammen.
  - Definieren Sie einen Konstruktor mit Parametern, und die Methoden print() der Klasse **Triangle.**
  - Die Klasse **Point** ist wie bisher auch gegeben.



KLASSE TRIANGLE

```
1  class Triangle
2  {
3      Point* _p_p1;
4      Point* _p_p2;
5      Point* _p_p3;
6  public:
7      Triangle(Point* p_1, Point* p_2,
8               Point* p_3){
9          this->_p_p1 = p_1;
10         this->_p_p2 = p_2;
11         this->_p_p3 = p_3;
12     }
13     void print()
14     {
15         _p_p1->print();
16         _p_p2->print();
17         _p_p3->print();
18     }
19 };
```

```
20 int main()
21 {
22     cout << "main beginnt" << endl;
23     // Punkte gehoeren zum Hauptprogramm
24     // und nicht zum Dreieck
25     Point p_1(-3, 0), p_2(0, 4), p_3(3, 0);
26     {
27         cout << "block beginnt" << endl;
28         Triangle t(&p_1, &p_2, &p_3);
29         t.print();
30         cout << "block endet" << endl;
31     }
32     cout << "main endet" << endl;
33     return 0;
34 }
```

HOCHSCHULE
ESSLINGEN

particify.hs-esslingen.de

7751 6136

Qrticify - QUIZ


```
1  #include <iostream>
2  using namespace std;
3
4  class Point {
5    double _x, _y;
6  public:
7    Point() {
8      _x = 0;
9      _y = 0;
10     cout << "StandardKonstruktor von: "
11          << _x << ", " << _y << endl;
12   }
13   Point(double x, double y) {
14     this->_x = x;  this->_y = y;
15     cout << "Konstruktor von: " << _x
16          << ", " << _y << endl;
17   }
```

```
18   ~Point(){
19     cout << "Destruktor von " << _x
20          << ", " << _y << endl;
21   }
22   double get_x() const { return _x; }
23   double get_y() const { return _y; }
24   void set_x(double x) { this->_x = x; }
25   void set_y(double y) { this->_y = y; }
26   void print() {
27     cout << "x= " << _x << endl;
28     cout << "y= " << _y << endl; }
29 };
```

Particify - QUIZ

```
1  class Triangle
2  {
3      Point* _p_p1;
4      Point* _p_p2;
5      Point* _p_p3;
6  public:
7      Triangle(Point* p_1, Point* p_2,
8               Point* p_3){
9          this->_p_p1 = p_1;
10         this->_p_p2 = p_2;
11         this->_p_p3 = p_3;
12     }
13     void print()
14     {
15         _p_p1->print();
16         _p_p2->print();
17         _p_p3->print();
18     }
19 };
```

```
20 int main()
21 {
22     cout << "main beginnt" << endl;
23     // Punkte gehoeren zum Hauptprogramm
24     // und nicht zum Dreieck
25     Point p_1(-3, 0), p_2(0, 4), p_3(3, 0);
26     {
27         cout << "block beginnt" << endl;
28         Triangle t(&p_1, &p_2, &p_3);
29         t.print();
30         cout << "block endet" << endl;
31     }
32     cout << "main endet" << endl;
33     return 0;
34 }
```

rticify - QUIZ: AUFLÖSUNG

```
1  class
2  {
3
4
5
6  pub
7
8
9
10
11
12
13
14
15
16
17
18
19 };
```

Ausgabe

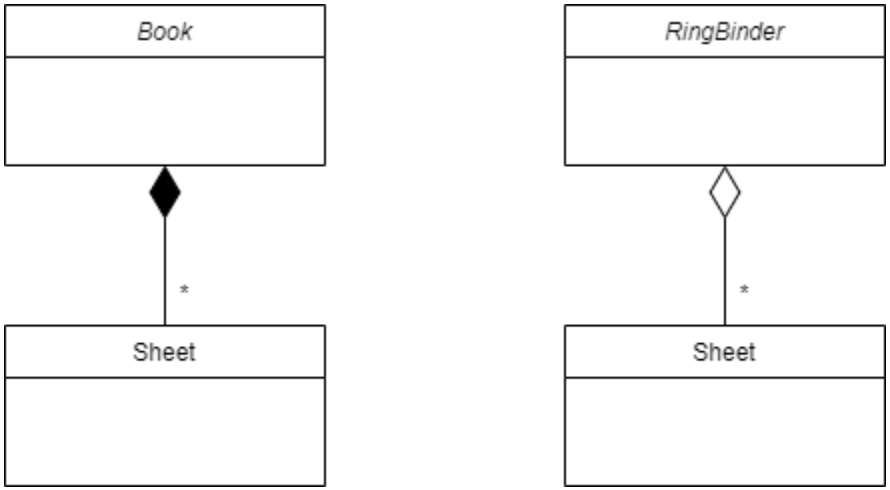
```
main beginnt
Konstruktor von: -3, 0
Konstruktor von: 0, 4
Konstruktor von: 3, 0
block beginnt
x= -3
y= 0
x= 0
y= 4
x= 3
y= 0
block endet
main endet
Destruktor von 3, 0
Destruktor von 0, 4
Destruktor von -3, 0
```

```
20 int main()
21 {
22     cout << "main beginnt" << endl;
23     // Punkte gehoeren zum Hauptprogramm
24     //und nicht zum Dreieck
25     Point p_1(-3, 0), p_2(0, 4), p_3(3, 0);
26     {
27         cout << "block beginnt" << endl;
28         Triangle t(&p_1, &p_2, &p_3);
29         t.print();
30         cout << "block endet" << endl;
31     }
32     cout << "main endet" << endl;
33     return 0;
34 }
```

FAZIT

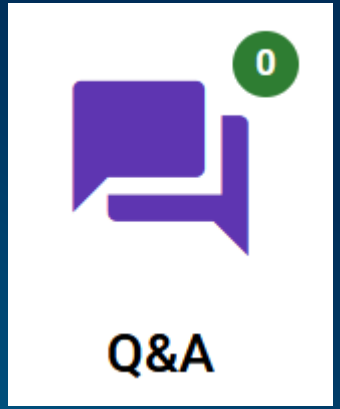
- **Komposition:**
  - Die Teilobjekte bilden ein unteilbares Ganzes.
  - Die Teilobjekte haben keine unabhängige Existenz.
  - Die Teilobjekte sind als Attribute eingebettet.
- **Aggregation:**
  - Die Teilobjekte können isoliert betrachtet werden.
  - Die Teilobjekte haben eine unabhängige Existenz.
  - Die Teilobjekte werden über Pointer oder Referenzen eingebettet.

WEITERES BEISPIEL



Hochschule Esslingen
University of Applied Sciences

FRAGEN?



HOCHSCHULE
ESSLINGEN

particify.hs-esslingen.de

7751 6136

Hochschule Esslingen
University of Applied Sciences

LABORAUFGABEN

AUFGABE 7

Themen:

Klassen, Klassenvariablen

Ihre Aufgabe:

Die Klasse 'Customer' soll folgende Attribute besitzen:

```
static int _s_count;
static int _s_id_generator;
std::string _name;
std::string _location;
int _age;
double _business_done;
int _transaction_count;
int _id;
```

Das Attribut '_business_done' gibt an, welcher Umsatz in Euro in einer Transaktion (d.h. einem Einkauf) gemacht wird. Die Variable '_id' gibt die Kundennummer an. Sie wird in Abhängigkeit der statischen Variablen '_s_id_generator' gesetzt. Die statischen Variablen '_s_count' und '_s_id_generator' werden automatisch inkrementiert, wenn ein Objekt der Klasse 'Customer' angelegt wird.

AUFGABE 7 (2)

Verfassen Sie
a) einen Konstruktor mit Default-Parametern für '_name', '_location' und '_age'.

b) einen Destruktor

c) die Methode 'do_business(double amount)', mit welcher '_business_done' um 'amount' erhöht und gleichzeitig das Atribut '_transaction_count' inkrementiert wird.

d) eine statische Methode 'get_s_count()', mit welcher die statische Variable '_s_count' zurückgegeben werden kann.

e) eine print()-Methode, mit welcher z.B. der unten angegebene Text ausgegeben werden kann:

```
Kunde Meier aus Esslingen (ID = 4, 28 Jahre) hatte 3 Transaktion(en) und 230 Euro Unsatz
```

AUFGABE 7 (3)

Das angegebene Hauptprogramm soll folglich die folgende Ausgabe erzeugen:

Ausgabe:

```
Anzahl Kunden: 3
Kunde Simon aus Heilbronn (ID = 2, 23 Jahre) hatte 3 Transaktion(en) und 633.99 Euro Umsatz
Kunde Michael aus Karlsruhe (ID = 3, 21 Jahre) hatte 1 Transaktion(en) und 199 Euro Umsatz
Kunde Claudia aus Nagold (ID = 4, 30 Jahre) hatte 1 Transaktion(en) und 1000 Euro Umsatz
Anzahl Kunden: 2
```

AUFGABE 8

Themen:

Komposition

Ihre Aufgabe:

Das Programm, das sie entwickeln werden, kann grafische Objekte (Punkte, Kreise, ...) als Objekte von Klassen darstellen. Sie können sich die Objekte als Datenrepräsentationen von grafischen Objekten in einem Zeichenprogramm vorstellen. Die grafischen Objekte sind definiert über Koordinaten und könnten (später) Eigenschaften haben wie z.B. eine Strichstärke und -art.

Sie sollen zwei Klassen definieren: Point und Circle.

(a) Die Klasse Point repräsentiert das geometrische Objekt Punkt.
(b) Die Klasse hat die Instanzvariablen '_x' und '_y' vom Typ double. Diese beiden Instanzvariablen repräsentieren die x- und y-Koordinaten eines Punktes.
(c) Die Klasse Circle repräsentiert einen Kreis.
(d) Sie hat die Instanzvariablen '_centre' vom Typ 'Point' und '_radius' vom Typ double. Die Variable '_centre' steht also für den Mittelpunkt des Kreises.

AUFGABE 8 (2)

(e) In beiden Klassen sollen die Instanzvariablen vor Zugriff von außerhalb der Klasse geschützt sein.
(f) Schreiben Sie für beide Klassen Konstruktoren mit geeigneten Defaultparametern.
(g) Schreiben sie get- und set-Methoden für alle Instanzvariablen.
(h) Die Instanzfunktion 'move(double dx, double dy)' verschiebt einen Punkt bzw. einen Kreis um die Werte 'dx' und 'dy' in x- bzw. y-Richtung.
(i) Beide Klassen sollen eine Funktion 'print' besitzen, welche die Koordinaten eines Punktes bzw. die Koordinaten des Mittelpunktes und den Radius eines Kreises auf dem Bildschirm ausgibt. Die Funktion 'print' besitzt einen Parameter vom Typ 'bool', der steuert, ob nach der Ausgabe ein Zeilenvorschub (endl) durchgeführt wird oder nicht. Der Defaultwert für diesen Parameter soll 'true' sein.
Die Ausgabe des gegebenen Hauptprogrammes ist unten angegeben:

Ausgabe:

```
Ausgabe 1:
(0, 0)
[(0, 0), 1]
Ausgabe 2:
(1.1, 2.2) == (1.1, 2.2)
[(1.1, 2.2), 3.3]
Ausgabe 3:
(2.1, 3.2)
[(3.1, 4.2), 3.3]
```

ZUSAMMENFASSUNG & AUSBLICK

ZUSAMMENFASSUNG

- Statische Klassenelemente
- Konstante Attribute, Objekte und Methoden
- Freundfunktionen + Freundklassen
- Komposition
- Aggregation

Hochschule Esslingen
University of Applied Sciences

DANKE



HOCHSCHULE
ESSLINGEN

particify.hs-esslingen.de

7751 6136